

3 - Design Patterns Descriptions (GOF)

1) Singleton

Intent :

The intent of the Singleton design pattern is to ensure that a class has only one instance and provides a global point of access to that instance.

Problem :

In software development, there are scenarios where having only one instance of a class is desirable. For example in a logging system, configuration manager, or database connector, having multiple instances could lead to issues such as inconsistent behavior, resource wastage, or conflicting state. The challenge is to enforce the single instance constraint while allowing global access to that instance.

Solution :

To solve this problem, the Singleton pattern defines a class that has only one instance and provides a way to access that instance from any point in the application. Typically, the Singleton class controls the instantiation process by ensuring that only one instance is created and providing a static method to access that instance.

Structure :

Singleton: Defines a static method that returns the instance of itself.

Instance: The single instance of the Singleton class.

Client: Any object or class that requires access to the Singleton instance.

Participants :

Singleton: The class that implements the Singleton pattern.

Client: Any class or object that needs to use the Singleton instance.

Consequences :

Controlled access to the sole instance:

Singleton pattern ensures that there is only one instance of the class and provides a global point of access to it.

Lazy initialization:

The instance is created only when it is first requested, which helps in conserving resources.

Global access:

Any part of the application can access the Singleton instance without needing to instantiate it.

Difficulties in subclassing:

Subclassing a Singleton can be tricky because the constructor is typically private, and the instance is accessed via a static method.

Implementation :

- 1. Private Constructor:** Ensure that the class has a private constructor to prevent instantiation from outside the class. This means that other classes cannot create instances of the Singleton class using the new keyword.
- 2. Static Instance:** Declare a static variable of the Singleton class type within the class. This variable will hold the sole instance of the class.
- 3. Static Method:** Provide a static method that acts as the global access point to the Singleton instance. This method should check if the instance exists, and if not, create it. It then returns the instance.

2) Observer

Intent :

The intent of the Observer Design Pattern is to define a one-to-many dependency between objects so that when one object changes state, all its dependents are notified and updated automatically.

Problem :

In software development, there are scenarios where multiple objects need to be informed about the changes that occur in one object's state. Without a proper design pattern, this could lead to tight coupling between the objects, making the system less flexible and harder to maintain.

Solution :

The Observer Design Pattern suggests defining a subject (also known as an observable) and multiple observers. Observers register themselves with the subject to receive updates when the subject's state changes. This decouples the subject from its observers, allowing for better flexibility and reusability.

Structure :

Subject: Maintains a list of observers, provides methods to register, unregister, and notify observers of changes.

Observer: Defines an interface with a method (e.g., `update()`) that the subject calls to notify observers of state changes.

Participants :

Subject: Interface or abstract class defining methods to add, remove, and notify observers.

Observer: Interface or abstract class defining the method that concrete observers must implement to receive updates.

Consequences :

1. Loose coupling between subjects and observers, making the system easier to maintain and extend.
2. Supports broadcast communication, where multiple observers can receive updates from a single subject.
3. Adding new observers does not require changes to the subject's code, promoting scalability and reusability.
4. Observers can be removed dynamically without affecting other parts of the system.

Implementation :

1. Define the Subject interface or abstract class with methods for adding, removing, and notifying observers.
2. Define the Observer interface or abstract class with a method to update its state based on changes in the subject.
3. Clients instantiate subjects and observers, register observers with subjects, and trigger state changes as needed.

3) MVC

Intent :

MVC is a design pattern aimed at separating concerns within an application, primarily separating the user interface (View) from the application logic (Model) and the control logic (Controller). Its intent is to provide a clear structure that promotes modularity, reusability, and maintainability of code.

Problem :

In traditional software design, user interface logic, application logic, and control logic are often intertwined, making it challenging to make changes or updates without affecting other parts of the system. This can lead to code that is difficult to understand, maintain, and scale.

Solution :

The MVC pattern divides an application into three interconnected components:

Model: Represents the application's data and business logic. It encapsulates the state of the application and provides methods to manipulate that state.

View: Represents the presentation layer of the application. It displays the data from the Model to the user and sends user input to the Controller for processing.

Controller: Acts as an intermediary between the Model and the View. It receives user input from the View, processes it (often by updating the Model), and updates the View accordingly.

Structure :

Model: Manages the application's data and business logic.

View: Displays the data to the user and forwards user input to the Controller.

Controller: Receives input, processes it, and updates the Model or View accordingly.

Participants :

Model: Represents the application's data and business logic.

View: Displays the data from the Model to the user and forwards user input to the Controller.

Controller: Receives user input from the View, processes it, and updates the Model or View.

Consequences :

Separation of Concerns:

Clear separation between user interface logic, application logic, and control logic, making the codebase more organized and maintainable.

Modularity:

Each component can be developed and tested independently, facilitating code reuse and easier collaboration among developers.

Flexibility:

Changes to one component (e.g., the View or Model) can be made without affecting the others, promoting easier evolution of the application over time.

Parallel Development:

Different teams or developers can work on different components simultaneously, speeding up the development process.

Implementation :

Model: Implement classes and methods to represent the application's data and business logic.

View: Implement classes and methods to display data to the user and capture user input.

Controller: Implement classes and methods to process user input, update the Model, and update the View accordingly. Ensure proper communication between the Model and View components.